Fort Hays State University

2025S_CSCI441_VA_Software Engineering

Mike Mireku Kwakye, Ph.D.

Software Engineering Project Full Report #1 and #2

Triple Tactics

Group F

Angel Escalera Aguirre, Iain Fehr, Juan Enriquez Mendes,
Seth Tharo Hour, Sinan Abdulraheem

March 23, 2025

URL: https://github.com/icfehr/Team_F_Project.git

Summary of Changes:

Iain - Updated User Effor Estimation and Conceptual Model

Iain- Added in user interface screenshots from implementation UI section now filled out with accurate information.

Iain - Update use case and update functional and nonfunctional requirements

Iain - Updated Change Log
Iain - Added and assigned Summary of Changes to Group members

Juan - Update section 8 with interaction diagrams with design patterns potentially publisher subscriber design

Juan - Revise use cases in section 4

Juan - Effort Estimation using Use Case Points

Angel - System Architechure needs slight revisions for updated model

Angel - Algorithms and data structures remain unchanged Network Elements Updated

Juan + Iain - Added References for pictures

Seth - Status unknown unable to assign tasks last communication 4/11

Sinan- Status unknown unable to assign tasks last communication 3/12

Work Assignment

Reference Key

R: Responsible	A: Accountable	C: Consulted	I: Informed

	Iain Fehr	Angel	Sinan	Juan	Seth
Problem Statement	R	R	R	R	R
Sub-Problems	R	A	A	A	A
Glossary	C	C	R	C	R
Business Goals	R	A	A	A	A
Functional Requirements	A	R	R	A	A
Nonfunctional Requirements	I	A	I	R	R
UI Requirements	R	I	R	A	R
Use Cases	R	R	I	I	I
User Interface Specification	R	I	R	A	A
System Architecture	I	I	R	R	A
Project Size Estimation	R	I	C	C	C
Conceptual Model	R	I	I	I	I
System Operation Contracts	I	C	I	R	I
Data Model and Persistent Storage	C	I	I	I	R
Interactions -Diagram	R	C	C	C	C

Class Diagram	C	R	A	R	A
Data Types and Operation Signatures	I	I	R	I	I
Class Traceability Matrix	I	I	I	R	C
Data Structures	I	R	R	I	R
Concurrency	C	R	C	C	C
Design of Tests	C	C	R	R	R
Project Management	R	C	C	C	C
References	R	R	R	R	R

Changes

This Evolved into Iain taking care of all user gameplay implementation Angel on server side implementation and Juan on asset creation.

Section 1: Customer Problem Statement of Requirements

Problem Statement

Modern mobile users often experience short periods of idle time—waiting in lines, taking breaks, or commuting—but lack engaging, time-efficient entertainment options. Many mobile games require lengthy sessions, making them unsuitable for quick play, while passive activities like social media scrolling offer little interactivity.

Imagine a world where the daily grind feels like a monotonous loop: wake up, work, go home, sleep. Your phone, a gateway to endless possibilities, sits idle, offering no escape from the humdrum routine. You yearn for a spark of excitement, a quick thrill to break the monotony. Remember those carefree high school days filled with laughter and friendly competition over card games like Triple Triad? Those memories feel like a distant dream now, lost in the whirlwind of adulthood.

But what if you could recapture that magic?

Group F is bringing back the beloved card game Triple Triad, reimagined for the modern world. Introducing Triple Tactics: your pocket-sized portal to quick, engaging card battles. Picture this: you're on a short break, waiting in line, or just winding down before bed. Instead of scrolling aimlessly, you pull out your phone or tablet and dive into a fast-paced Triple Tactics match. Each game is designed to last no more than 45 seconds, making it the perfect bite-sized entertainment for those fleeting moments of free time.

To address the issue of lacking engaging, time-efficient entertainment options, Triple Tactics reimagines the classic Triple Triad card game for modern mobile platforms. It provides:

Fast-paced gameplay, with each match lasting under 45 seconds.
A simple, intuitive rule set, learnable in under two minutes.
A collectability system, adding depth and long-term engagement.
Multiplayer functionality, enabling remote play with minimal latency.

Triple Tactics offers a simple, intuitive rule set that can be learned in under two minutes. Whether you prefer to read the instructions or jump right into the action, you'll be mastering the

basics in no time. And the fun doesn't stop there! We're building a future for Triple Tactics that includes exciting collectability features, adding depth and replayability to the game.

Triple Tactics isn't just a game; it's a way to transform those empty moments of time into fun and friendly and, most importantly, quick competitive moments. It's a way to reconnect with the joy of card games and inject a little bit of excitement back into your day. So, ditch the mindless scrolling and embrace the quick, strategic thrills of Triple Tactics. Your daily routine deserves a little bit of magic. Tired of the daily grind? Your phone, a gateway to endless possibilities, sits idle, offering no escape from the monotony. Remember those carefree high school days, filled with laughter and friendly competition over card games like Yu-gi-oh, Magic the Gathering, and the ever-classic poker? We're bringing that magic back with Triple Tactics!

Triple Tactics is the beloved card game Triple Triad from the Final Fantasy series, reimagined for quick entertainment. It's your pocket-sized portal to quick, engaging card battles. On a short break, waiting in line, or just winding down before bed? Instead of scrolling aimlessly, dive into a fast-paced Triple Tactics match. Each game is designed to last no more than 45 seconds, making it the perfect entertainment for those fleeting moments of free time.

Triple Tactics offers a simple, intuitive rule set that can be learned in under two minutes. Whether you prefer to read the instructions or jump right into the action, you'll be mastering the basics in no time. And the fun doesn't stop there! We're building a future for Triple Tactics that includes exciting collectability features, adding depth and replayability to the game.

Triple Tactics isn't just a game; it's a way to transform those empty pockets of time into fun and friendly competition. It's a way to reconnect with the joy of card games and inject a little bit of excitement back into your day. So, ditch the mindless scrolling and embrace the quick, strategic thrills of Triple Tactics!

Technical Challenges:

- The development of Triple Tactics involves key software engineering challenges, including:
- Card mechanics implementation – Designing card interactions, placement rules, and win conditions.
- User interface design – Creating a responsive, intuitive layout for fast interactions.

- Game state management – Ensuring seamless transitions between turns and tracking match progress.
- Multiplayer optimization – Implementing real-time matchmaking with minimal lag.
- By addressing these challenges, Triple Tactics aims to fill a market gap for fast, skill-based, competitive gaming, providing a lightweight yet engaging alternative to existing mobile entertainment.

Decomposition into Sub-problems

- **Card Mechanics**: The basic rules of the card game mechanics (cards, power, location, etc.)
- **User Interface Design**: Simple and intuitive interface that fits within the quick time frame of play.
- **Game Flow**: Seamless transitions between turns, tracking of game state, and win conditions.
- **Collectability System**: Allowing players to collect and trade cards over time.

Multiplayer Functionality: Connecting players for remote play with minimal lag.
Example of a game board design layout with each X being a card

```
  +---+---+---+
| X | X | X |
  +---+---+---+
| X | X | X |
  +---+---+---+
| X | X | X |
  +---+---+---+
```

Glossary

- **Deck**: The pool of cards not yet drawn into your hand, built before the match.
- **Hand**: The cards you currently hold are available for play.
- **Cards in play**: The cards are placed on the board.
- **Color**: Ownership of the cards on the board, where Player One is Red and Player Two is Blue.
- **Power**: The strength of the side of a card when interacting with another card.

- **Location**: The position of a card on the 3x3 grid, occupying one of the nine available slots.

Ruleset: A set of instructions that governs how the game is played, such as card placement, card ownership, and winning conditions.

Section 2: Business Goals and System Requirements

Business Goals

- **Quick and Engaging Gameplay:** The primary attraction of *Triple Tactics* is that each game can be played in under 45 seconds. This makes it ideal for casual gaming during brief moments of downtime, which appeals to a broad audience—whether they're commuting, waiting for an appointment, or taking a break at work. This quick play style encourages repeated sessions, leading to high user retention.
- **Collectibility and In-game Rewards**: *Triple Tactics* offers a collectible card system similar to popular games like *Hearthstone* or *Gacha* systems, where players can collect rare cards, trade, or even sell them in the future. This element adds long-term value for players, keeping them engaged over time. Investors will find this appealing, as collectibility can lead to microtransactions and in-game purchases, a proven strategy for monetization in mobile games.
- **Potential for Expansion**: The game's core mechanics allow for frequent updates, including new cards, game modes, and challenges. This constant refresh keeps players engaged and provides opportunities for ongoing revenue generation through app updates, DLCs, or events.

- **Fun, Short, and Collectible**: Provide a quick, easy-to-learn game that players can enjoy in short bursts while offering opportunities for card collection.

- **Accessible Anywhere**: Enable gameplay on mobile devices with low setup time and minimal learning curve.

Engagement: Include features that encourage repeat play and in-game purchases or rewards (if applicable).

Functional Requirements

Functional Requirements for Triple Tactics

1. Card Mechanics:
    - Card Types: Cards should have defined types, each with a unique power rating on four sides (top, right, bottom, left). Each side of the card must be associated with a power value to interact with opposing cards.
    - Card Placement: Players can place cards on a 3x3 grid. Once placed, cards interact with neighboring cards to determine the winner of that particular slot.
    - Power Comparison: When two cards face each other, the higher power on the side facing the opponent's card wins, and ownership of the card changes accordingly.
    - Win Condition: The player who controls the most cards on the grid at the end of the game wins. If a card is surrounded by opposing cards on all four sides, it gets flipped.
2. Game Flow:
    - Turn-Based Gameplay: Players take turns placing one card at a time on the grid. The game ends when all cards have been placed or when one player has won.
    - Timer: Each match should be completed in under 45 seconds. A countdown timer will be displayed, ensuring fast-paced gameplay.
    - Match Setup: Each player is dealt five cards from their deck at the start of the match. Cards are drawn randomly from the deck.
    - Game State: The system must track which player owns which cards on the board and update the game state after each turn.
    - End of Game: At the end of the match, the game must declare a winner and display the result (e.g., “Player One Wins!”). The board should highlight which cards belong to which player.
3. User Interface Design:
    - Minimalist Design: The UI should be clean, showing only necessary elements—game board, player stats, card details, and the timer.
    - Card Display: Cards in the player's hand should be displayed in a scrollable area at the bottom of the screen, with each card showing its power values.
    - Game Board: The 3x3 grid should be clearly visible, with slots that update in real time to show the current state of play.

- Indicators: Player turns, remaining time, and player scores should be visible throughout the game.
- Animations: When a card is placed or flipped, there should be animations to provide clear feedback to the player. This includes flipping cards and showing a winning player at the end.

4. Collectability System:
    - Card Collection: Players can earn new cards by winning games, completing challenges, or purchasing packs. Cards have rarity levels (common, rare, epic, etc.).
    - Card Packs: Cards can be purchased as part of a pack that contains random cards. This creates a collectible experience and can generate revenue through in-game purchases.
    - Trade/Marketplace: Players should be able to trade cards with other players in a simple marketplace, allowing them to obtain cards they need for their deck.
    - Card Collection Progress: Track players' progress in collecting all available cards, with rewards for completing sets or achieving milestones.
5. Multiplayer Functionality:
    - Remote Play: Players should be able to compete against others remotely, via online matchmaking or inviting friends directly.
    - Matchmaking: The system should match players based on their skill level, ensuring fair and competitive matches.
    - Latency Management: The game should minimize lag and ensure smooth gameplay even with remote matches.
    - Leaderboards: Players' rankings based on wins, cards collected, and game stats should be displayed.
6. Accessibility Features:
    - Simple Controls: Ensure that the game can be played using simple touch gestures or taps, suitable for mobile devices.
    - Colorblind Mode: Implement a colorblind mode that uses different symbols or patterns to represent each player’s cards, ensuring accessibility for all players.
    - Quick Tutorial: Include an easy-to-understand tutorial for first-time players, explaining the basic mechanics of the game

Nonfunctional Requirements

1. **Requirements in performance**

- Duration of each match: Must be 45 seconds or less so that sessions can be short and not too long.
- Frame rate: 30 fps is a perfect way to start off. We can tweak later if needed.
- Loading time: We need the game to launch within a few seconds. Anywhere from 3 to 5 seconds should do the trick.
- Latency: Any lag should be maintained well under 100 ms so that the game isn't super laggy and unenjoyable.

2. **Security**

- User/Customer Privacy: ALL information must be stored securely such as personal information, logins, etc.
- Hack patches: Make sure that the game is flawless when it comes to playing it and not allow people to have the ability to cheat by using hacks or bots.

3. **Usability**

- Understanding of the game: Users need to know and understand at least the basics of the game within a few minutes. A quick and short tutorial should do the trick.
- Help/Support: Within the app, there should be a feature that provides the answer to many common questions such as an FAQ.

4. **Reliability**

- Uptime: The system should have 99.9% of uptime and 0.01% of downtime, meaning that the system is up and running 99.9% of the time and 0.01% of the time.
- Recovery: Sometimes, the game might crash. We first need to make the game so well so that it barely ever crashes or has errors. Second of all, in case that happens, we need to make sure that the game recovers properly and the error is fixed right away. This way, the user can resume their session/match or restart it without any issues or losing progress. Therefore, the user doesn't experience the frustration of going back and seeing that their session/match lost progress.
- Data Backup: Data that the player achieves such as wins, trophies or a card collection must all be able to be backed up regularly. Eventually, in case of a system failure, everything can easily be recovered with a recovery process.

5. **Maintenance**

- Bug fixes and updates: We should be able to deploy bug fixes and updates to this system. That way the system doesn't become outdated and thankfully keeps running smoothly and as expected as time passes by.

6. **Compatibility**

- OS: Our game must be compatible with any version of the operating system in any phone. We might want to go ahead and make it compatible with a computer too.
- We should implement the game so that it is able to be played on devices with at least 2GB of RAM.

User Interface Requirements

- The user interface must present the 3x3 grid clearly, with each player's hand easily accessible.
- All cards must be easily readable to the player
- Players must be able to recognize the strength of each side of a given card at any given time.
- Players must be able to select and place cards in available slots quickly.
- The game should provide clear feedback on ownership changes (e.g., color changes on cards) and display the winner at the end of the match.
- Players Should be able to easily navigate and understand all user menus in allowance to basic navigation for personal deck building and starting a game with other players.
- All menus and interface should be simple enough that any player can track the status of an ongoing game at a glance
- The number of total squares controlled must be easily identifiable

Section 3: Functional Requirement Specification and Use Cases

Stakeholders

- **Players (End Users)** - Individuals who engage with the game for entertainment.
- **Game Developers** - The development team responsible for coding and maintaining the game.
- **Game Moderators** - Ensuring fair play and managing in-game reports.

Actors and Goals

Initiating Actors:

1. **Player**:
    - Start a new game.
    - Place cards on the board.
    - Competes against an opponent.
    - Collects and trades cards.
    - View their card collection.
    - Goal: Engage in and win matches.

Participating Actors:

1. **Game System:**
    - Manages game state.
    - Determines win conditions.
    - Updates player statistics.
    - Provides real-time feedback (e.g., color changes in ownership).
2. **Opponent (Computer/Bot):**
    - Competes against the player in the match.

Use Cases

Use Case 1: Start a New Game

Primary Actor: Player
Goal: Begin a new match against either a computer opponent or a remote player.

Preconditions:

- The player has opened the game.
- The player has selected a mode (Solo or Multiplayer).

Postconditions:

- A new game session begins.

- Player is matched with an opponent (computer or another player).
- Player receives a random set of 5 cards.

Flow of Events:

1. The player selects the option to start a new game.
2. The system presents the player with two options: play against a computer or match with an online opponent.
3. The system displays the loading screen while matching with an opponent.
4. Once the opponent is found (or the computer is chosen), the system sets up the game board and assigns 5 cards to each player.
5. The game screen loads, and the match begins with the player taking their first turn.

Alternative Flow:

- If no opponent is found, the game will notify the player and offer to retry or play against a computer opponent.

Use Case 2: Place a Card on the Game Board

Primary Actor: Player
Goal: Place a card on the 3x3 grid during their turn.

Preconditions:

- The player has started a match.
- The player has a card in their hand to play.
- The player is on their turn.

Postconditions:

- The card is placed on the board.
- The game system updates the board with the new card placement and evaluates the change in card ownership.

Flow of Events:

1. The player selects a card from their hand.
2. The system highlights the available empty slots on the board.
3. The player selects a slot to place the card.
4. The system places the card in the selected slot and compares the card’s power values with neighboring cards.
5. If the player's card is stronger on a given side, the card(s) on the opponent’s side are flipped to the player’s color.
6. The system updates the game state to reflect ownership changes and determines if the game has ended.
7. The system advances the turn to the opponent.

Alternative Flow:

- If the player tries to place a card in an already occupied slot, the system will prompt them to choose a different slot.

Use Case 3: End Game and Display Results

Primary Actor: Game System
Goal: Determine the winner of the game and display the results.

Preconditions:

- All cards have been placed on the board, or one player has won by controlling more than half of the cards on the board.

Postconditions:

- The winner is declared.
- The system provides feedback to the player about the outcome of the match.

Flow of Events:

1. The system checks the board to count the number of cards controlled by each player.
2. If all cards have been placed, or if one player has flipped all but one of the opponent's cards, the game ends.

3. The system announces the winner (e.g., "Player One Wins!").
4. The system records the win and updates player statistics (e.g., wins, losses).
5. The player is presented with the option to return to the main menu or play again.

Alternative Flow:

- If there is a tie (an equal number of cards controlled by both players), the game declares a draw and provides an option to play again.

Use Case 4: Collect Cards and Build Deck

Primary Actor: Player
Goal: Collect new cards and build a custom deck for future matches.

Preconditions:

- The player has completed matches and earned rewards.
- The player is on the main menu or card collection screen.

Postconditions:

- The player has acquired new cards, either by winning games, purchasing packs, or trading with other players.
- The player has the option to modify their deck.

Flow of Events:

1. The player selects the "Card Collection" option from the main menu.
2. The system displays the player's current collection and available packs for purchase.
3. The player opens a new pack or receives rewards from completing challenges.
4. New cards are added to the player’s collection.
5. The player can select cards to add to their deck for future games.

Alternative Flow:

- If the player does not have enough in-game currency or resources, they will be prompted to purchase more or continue playing to earn cards.

Use Case 5: Play Against a Remote Opponent

Primary Actor: Player
Goal: Compete in a multiplayer match against a remote player.

Preconditions:

- The player is connected to the internet.
- The player has selected "Multiplayer" mode from the main menu.

Postconditions:

- A multiplayer game session begins, where the player competes against a remote player.

Flow of Events:

1. The player selects "Multiplayer" mode from the main menu.
2. The system checks the player's internet connection and begins the matchmaking process.
3. Once a remote opponent is found, the system loads the game and assigns each player 5 random cards.
4. The game proceeds as described in Use Case 2, where both players take turns playing cards on the 3x3 grid.
5. The game continues until a winner is determined, and the system updates the player's statistics accordingly.

Alternative Flow:

- If the system cannot find a match, the player will be notified and given the option to retry.

Use Case 6: Trade Cards with Other Players

Primary Actor: Player
Goal: Trade or sell cards with other players.

Preconditions:

- The player has at least one card they are willing to trade.
- The player is connected to the multiplayer marketplace system.

Postconditions:

- Cards are traded between players.
- The player’s card collection is updated accordingly.

Flow of Events:

1. The player selects "Trade" from the card collection menu.
2. The system displays a marketplace where players can list cards for trade or sale.
3. The player selects a card they wish to trade and offers it for a desired card.
4. The system checks if the trade offer is valid (i.e., both players agree to the trade).
5. If agreed, the system processes the trade and updates both players' collections.

Alternative Flow:

- If the trade offer is not accepted, the system will notify the player and return them to the marketplace screen.

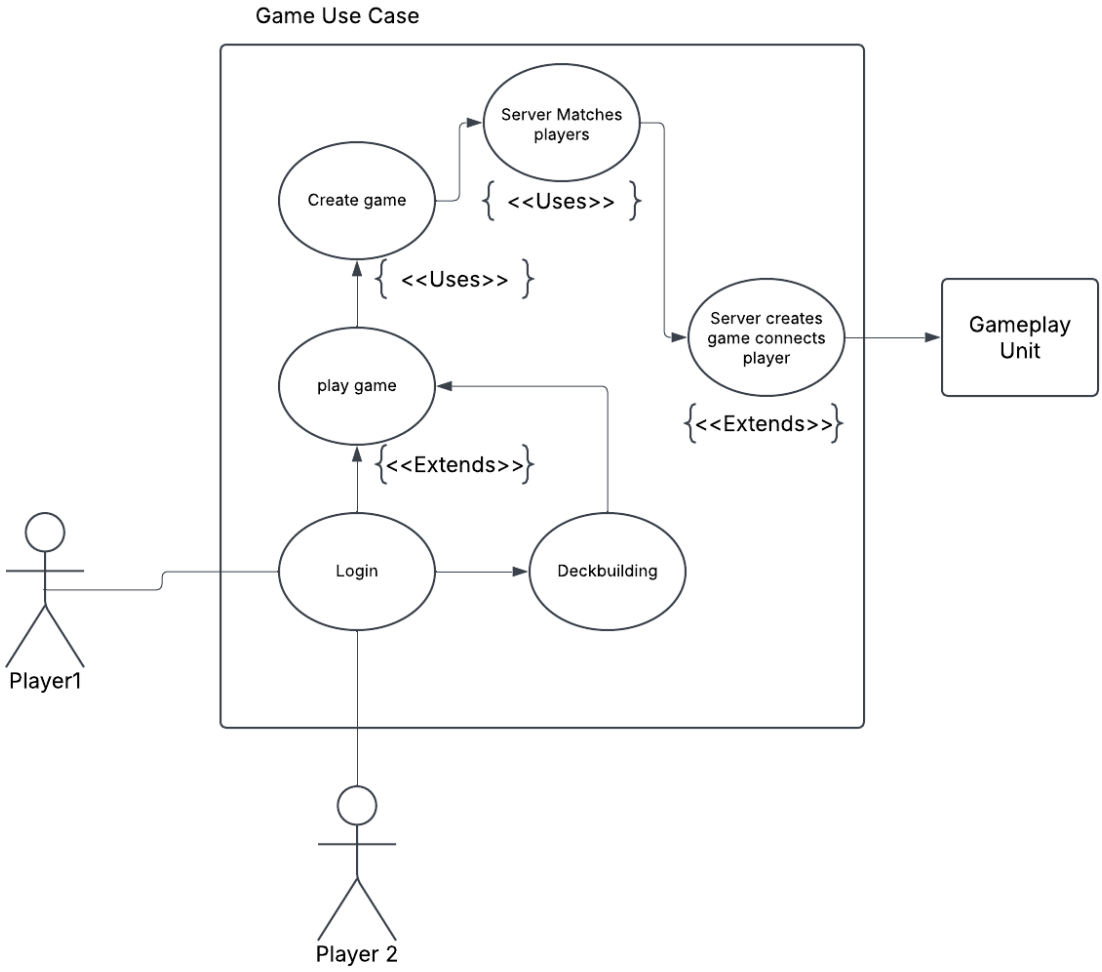
These use cases outline the fundamental interactions that players will have with Triple Tactics, from starting a game to collecting cards and competing against others. By addressing these functional requirements, the game can provide a streamlined, engaging experience for players while maintaining competitive fairness and long-term engagement.

Casual Description

1. Start a New Game
    a. Description: A player selects the "New Game" option to start a match.
    b. Responds to Requirements: REQ-1, REQ-3
2. Place a Card
    a. Description: A player selects a card from their hand and places it on the board.
    b. Responds to Requirements: REQ-2

3. Determine Game Winner
    a. Description: At the end of the game, the system calculates the winner based on card ownership.
    b. Responds to Requirements: REQ-2
4. Collect and Trade Cards
    a. Description: Players can collect new cards and trade with others to build their deck.
    b. Responds to Requirements: Functional Collectibility System
5. View Card Collection
    a. Description: Players can browse their owned cards and manage their deck.
    b. Responds to Requirements: User Interface Design

Use Case Diagram



Traceability Matrix (1) - System Requirements to Use Cases

	A	B	C	D	E	F
1	Use Case	Requirement	Priority	Designed	Testing	Deployment
2	Login	Creates and holds player unique ID and information	0	No	Not started	Not started
3	Play Game	Menu system for player choice starts matchmaking process	1	No	Not started	Not started
4	Deck_builder	Deck builder for player progression and deck choice	2	No	Not started	Not started
5	Create Game	Essential component starts the create game process	1	In-Progress	Not started	Not started
6	Server_matching	Essential component contacts server and executes player match making	1	No	Not started	Not started
7	game_creation	Essential component creates gameplay room for players to interact	1	No	Not started	Not started
8	gameplay_unit	Essential component room for player connection and interaction	0	No	Not started	Not started
9						

Fully-Dressed Description

A. Use Case: Start a New Game

Actors: Player, Game System

Preconditions: Player has launched the game and is on the main menu.

Trigger: Player selects "New Game."

Basic Flow:

1. The player selects "New Game."
2. The system displays the 3x3 grid and player's hand.
3. The game begins with a bot player starting.
4. The system waits for the player's first move.

Postconditions: The game state is initialized, and the first move is expected.

B. Use Case: Place a Card

Actors: Player, Game System

Preconditions: The player has started a game, and it is their turn.

Trigger: Player selects a card and a position on the board.

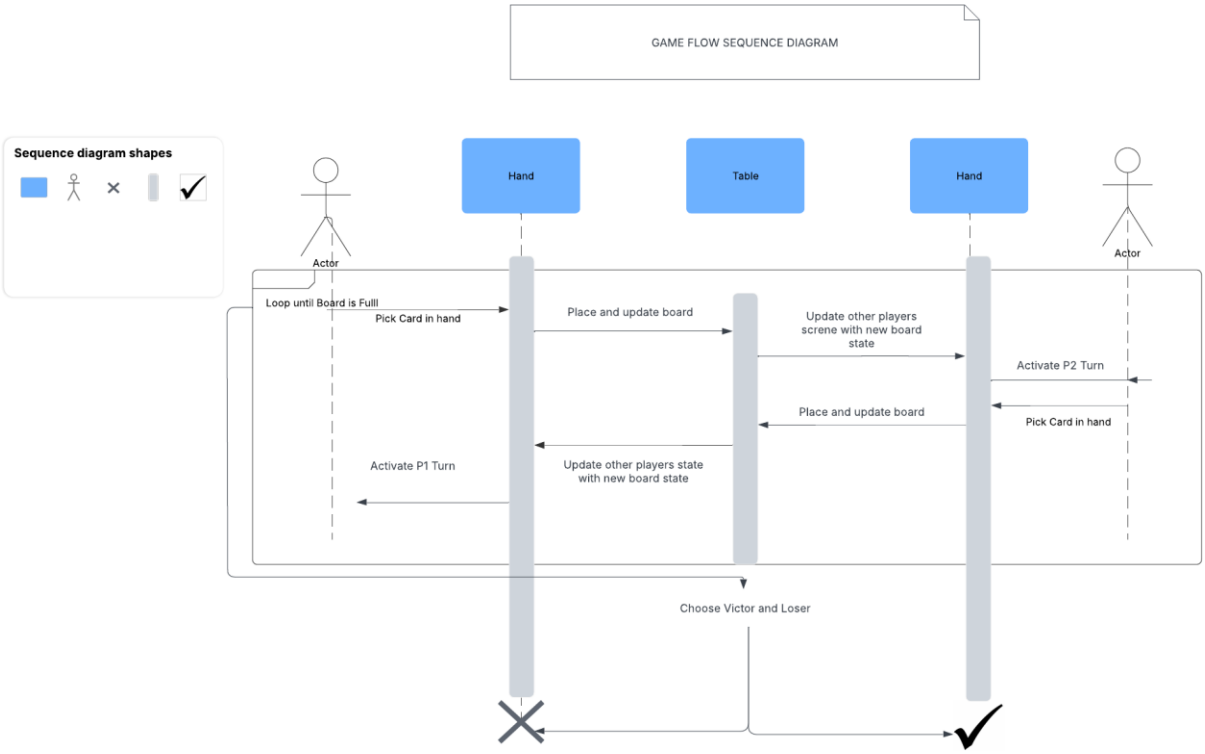
Basic Flow:

1. The player selects a card from their hand.
2. The player taps a valid position on the board.

3. The system places the card and evaluates ownership changes.
4. The system updates the board state and shifts turns.

Postconditions: The game state updates with the new card placement.

System Sequence Diagrams



Section 4: Updated User Interface Specification

Preliminary Design

The user interface for *Triple Tactics* is designed for quick, intuitive gameplay with minimal navigation effort. Below is a step-by-step breakdown of the UI flow for key use cases.

Start a New Game

1. **Main Menu**: The player sees three options:
    - **New Game**
    - **Card Collection – CUT**
    - **Settings**
2. **Game Initialization**: Upon selecting “New Game,” the screen transitions to a 3x3 grid game board. The player's hand appears originally intended to be on the bottom of the screen is now on the left and right sides of the screen.
3. **Match Start**: A prompt displays, indicating if the player or the AI goes first.

Place a Card

1. The player taps a card from their hand.
2. The available slots highlight valid placements.
3. The player taps a slot to place the card.
4. The system updates the board, evaluates ownership changes, and shifts the turn to the AI.
5. The AI makes its move automatically.

Determine Game Winner

1. Once all nine slots are filled, the system evaluates the board based on card ownership.
2. A message displays the winner and the final board state.
3. The player sees a summary screen with the option to start a new game or return to the main menu.

Collect and View Cards

1. The player navigates to the “Card Collection” menu.

2. They can browse, filter, and inspect individual cards.

User Effort Estimation

Start a New Game

- **Clicks/Taps**: 2 (Main Menu → New Game)
- **Navigation Effort**: Low

Place a Card

- **Clicks/Taps**: 3 (Select Card → Choose Slot → Confirm Placement)
- **Navigation Effort**: Low

Determine Game Winner

- **Clicks/Taps**: 1 (Acknowledge Results)
- **Navigation Effort**: Minimal

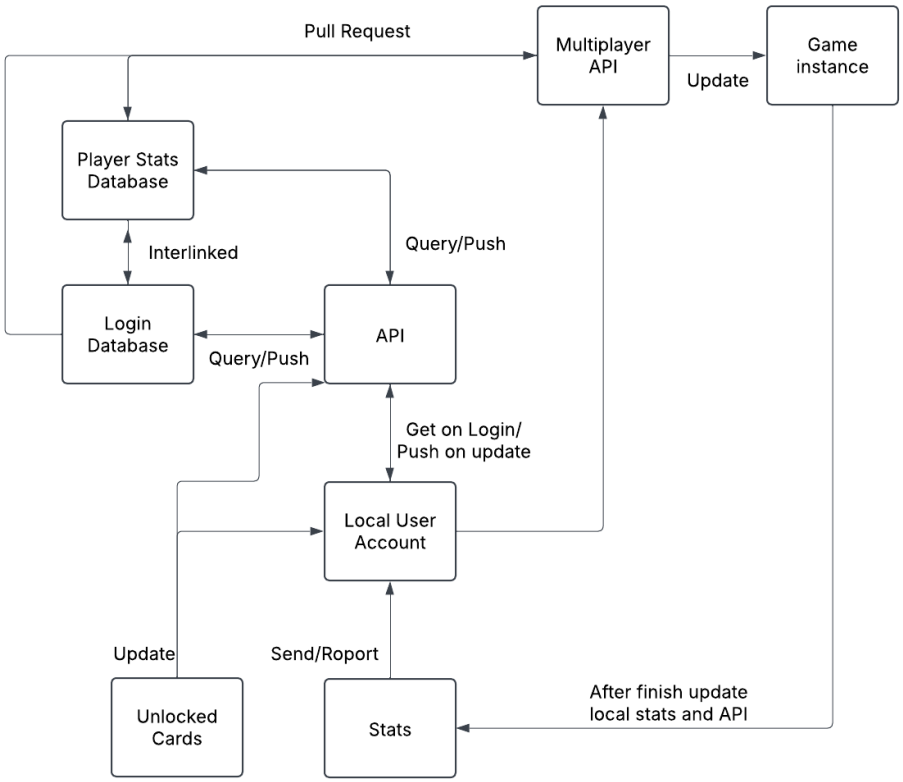
View Card Collection

- **Clicks/Taps**: 2+ (Navigate to Collection → Browse Cards)

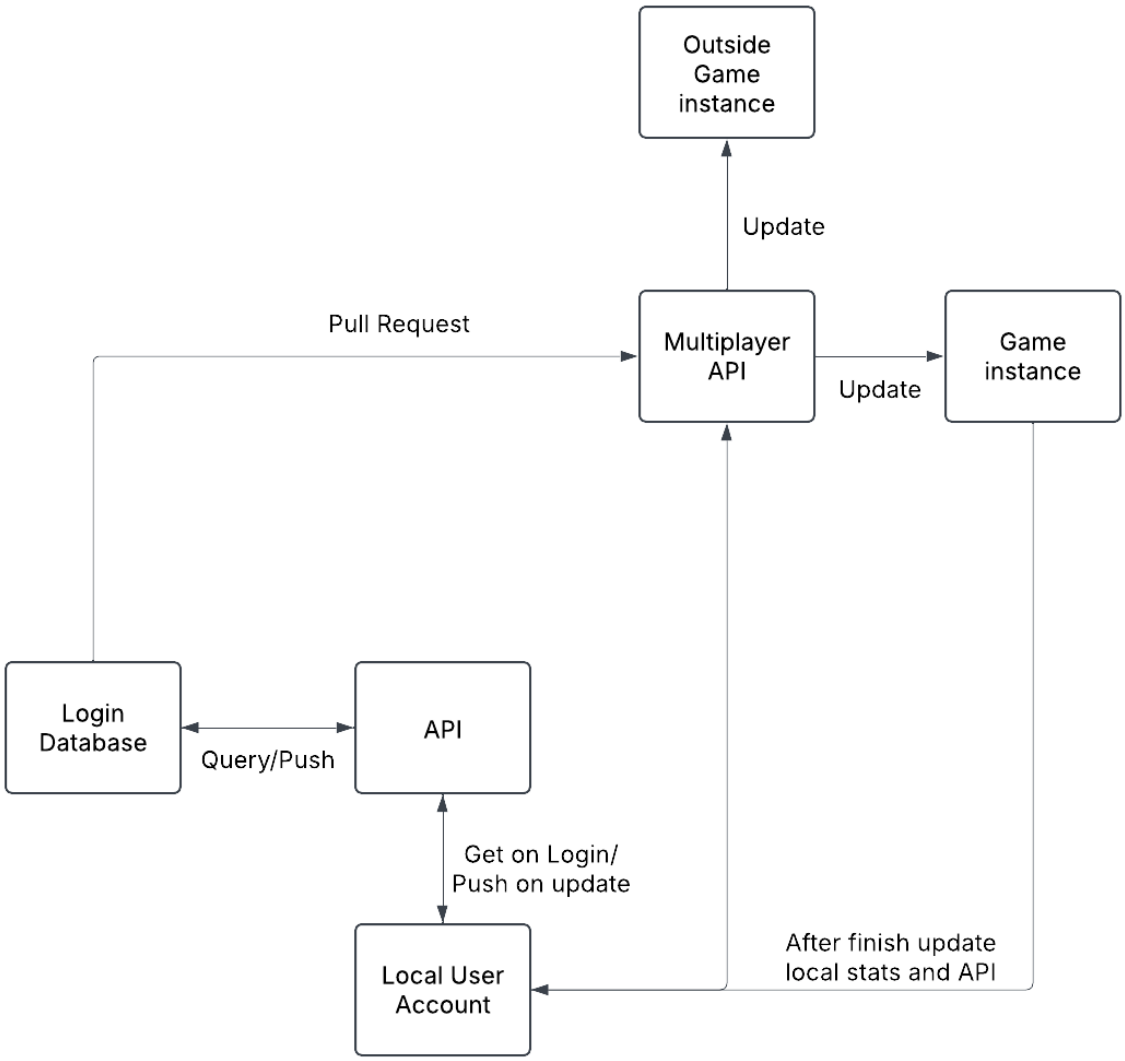
Navigation Effort: Low

Section 5: Analysis and Domain Modeling

Original Concept Model



Conceptual Model Update



Update:

As the project has evolved and circumstances around ability to complete based on scope arose the core concept of the project evolved to become more streamlined pruning away features that while nice in concept would not have been able to be executed within the timeframe with some group members not participating in the creation of the project execution. Because of this unnecessary features were pruned from the concept model to only the bare necessities that would allow the game to function and interact between instances. This is symbolised by the game instance checking the local user account then querying the login database before connecting to the pull request of the multiplayer api which connects the two game clients and will mirror each move on both game clients. Nothing outside the core components required for the model to function remain, like stats unlocked cards or a second user account database for tracking player stats. These proved to be unnecessary for base function and with a lack of support they were cut from the final iteration of the project demo.

Concept Definitions

Concept Name	Type Doing/Knowing	Responsibility
Login Database	Knowing	Collects and contains all the login player data for everyone who has logged in
Player Stats Database	Knowing	Collects and contains all of what each player has unlocked and a win loss record for each player
API	Doing	Securely connects and updates databases to fetch and store information
Local User Account (Local Database)	Knowing	Contains all the local player data and connects with the API to connect to the server databases
Stats(Local Database)	Knowing	Contains all the local player stats, and will also contain player settings and local records. After the update will pass along data to the Account to update server databases
Unlocked Cards (Local database)	Knowing	Contains a record of all unlocked cards the player can use on a local level.
Multiplayer API (API)	Doing	Securely connects players

		using a matchmaking system and connects players to the game instance
Game Instance (Web App)	**Doing**	**A room that is created to securely connect players for playing and update local player stats after they are destroyed.**

Association of Definition

Concept	Attribute	Description
API ↔ Login Database	Stores Local user account information on the Server	Database Query
API ↔ Player Stats Database	Stores Player stats on the server	Database Query/Push
Local User Account↔API	Creates user profile and allows for offline play. Stores all local information and settings	Database Query/Update
Player Stats Database ↔ Login Database	Uses Login information to Verify the correct player stats information and gives a layer of redundancy	Create/Verify
Local User Account -> Multiplayer API	Updates Multiplayer Api with player information	Create/Verify
Login Database ↔ Player Stats Database ↔ Multiplayer API	Interconnects to pull the correct players, player information, and information into a game instance. Also Protects against cheating by verifying storage information	Create/Verify/Send/Retrieve
Multiplayer API-> Updates Game instance	Uses Player information to update game instance room	Send
Game instance-> Stats	After a game is finished but before the instance is destroyed, update the local player stats.	Report/Update

Attribute Definitions

Concept	Attribute	Description
API	DB connection	Connects information from local storage to update server API databases
Multiplayer API	Db connection User ID User stats DB User Name User Network information	Connects all information from databases while linking players and creating a new game room instance
Web App	User ID 1 User ID 2 User 1 stats User 2 stats User 1 Name User 2 Name User 1 Network Tunnel User 2 Network Tunnel	The user interface element is designed to connect information together and link the information given for use. ID Stats and Name are pulled from the Database and display the available information. Network Tunnel is designed to link players 1 and 2 Together
Local Storage	User ID User Name User Stats User collection	Player Unique User ID Player Username Player Stat Profile Player local collection information
Instance	Game instance	Generated instance to host the web app and designed to connect players together for play. Hosts web app and is destroyed on game completion

Traceability matrix (2) - Use Cases to Domain Concept Objects

Use Case	Webapp	User Account	Local	API	Database
View Unlocked Cards		X	X		X
View Player Stats		X	X		X
Retrieve information			X	X	
Login Database	X				X
Player Stats Database	X				X
Game Instance	X			X	
Local Account inforation		X	X		

System Operation Contracts

Matchmaking

- **Inputs**:
    - Player's skill level (optional based on ranking system).
    - The available player pool for matchmaking.
- **Outputs**:
    - Match status (“Match Found” or “Searching”).

    - Opponent data (opponent's player ID, skill level).
- **Postconditions**:
    - A match is established if an opponent is found.
    - If no opponent is found, the system will attempt to find another opponent.
- **Exceptions**:
    - Timeout (if an opponent cannot be found after a certain period).

---

3. Starting a Game

Operation: `Start Game`

- **Preconditions**:
    - Both players (player 1 and player 2) must have completed the matchmaking process.
    - Players must have their cards and decks ready.
    - Both players must be connected to the internet.
- **Inputs**:
    - Player's deck of 5 cards.
    - Opponent's deck of 5 cards.
- **Outputs**:
    - Game state initialization, including the empty 3x3 board grid.
    - A countdown timer for the match to start.
    - Player's hand of 5 cards displayed on the UI.
- **Postconditions**:
    - Both players are in the game, with cards drawn from their respective decks.
    - The 3x3 grid is ready to accept card placements.
- **Exceptions**:
    - Network loss (unable to synchronize the game).
    - Incorrect deck state (e.g., not having 5 cards ready for both players).

---

4. Card Placement

Operation: `Place Card`

- **Preconditions**:
    - The game must be in progress.
    - The player must have a card in their hand.
    - The selected position on the 3x3 grid must be empty.
    - It must be the player's turn.
- **Inputs**:
    - The card to be placed (with power values).
    - The target position on the 3x3 grid (row, column).
- **Outputs**:
    - The selected card is placed on the grid at the given position.
    - The game state is updated, and adjacent cards are compared to determine if any cards are flipped.
    - The turn transitions to the opponent after the move.
- **Postconditions**:
    - The card is placed and interacts with neighboring cards based on its power.
    - The game state is updated to reflect the new board layout and ownership.
- **Exceptions**:
    - Invalid placement (e.g., card is placed outside the grid or on an already occupied slot).
    - Power comparison failure (e.g., no valid card to flip).
    - Network failure (unable to update the server with the move).

---

5. End of Game

Operation: `End Game`

- **Preconditions**:
    - The game must be in progress.
    - All cards must be placed, or a player has won by flipping all the opponent's cards.
- **Inputs**:
    - The final grid state, including all cards in play.
- **Outputs**:
    - Display of the winner (e.g., "Player 1 Wins" or "Player 2 Wins").

    - Updated player stats (wins, losses, card collection progress).
    - Potential rewards (e.g., new cards, experience points).
- **Postconditions**:
    - Game results are recorded and stored in the player's profile.
    - The match results are reflected in the leaderboard and any rankings.
- **Exceptions**:
    - Unexpected server disconnection (match ends prematurely, and data may be lost).

---

6. Card Collection

Operation: `Earn New Cards`

- **Preconditions**:
    - The player must have completed a match or challenge that awards cards.
    - The player must have space in their collection (if applicable).
- **Inputs**:
    - Card reward data (new card type, rarity).
- **Outputs**:
    - The card is added to the player's collection.
    - A notification is displayed to the player about the new card earned.
- **Postconditions**:
    - The player's collection is updated with the new card.
    - The player can now use the new card in future games.
- **Exceptions**:
    - Database error (unable to update the collection).
    - Card duplication (e.g., the player already owns the card).

7. Leaderboard Update

Operation: `Update Leaderboard`

- **Preconditions**:
    - The game must have ended with a winner.
    - The player must have completed a valid match.

- **Inputs**:
    - Match results (winner, score, number of cards collected).
    - Player's ranking progress (new points or rank).
- **Outputs**:
    - Leaderboard data is updated to reflect the new rankings.
    - Player's position is recalculated based on the new match result.
- **Postconditions**:
    - The player's ranking is updated on the leaderboard.
    - The updated leaderboard is accessible by all players.
- **Exceptions**:
    - Invalid match result.

Data Model and Persistent Data Storage

Database Schema

A relational database (PostgreSQL) is used for structured data storage.

Schema Overview:

· **Players** (id, username, rank, deck_id)

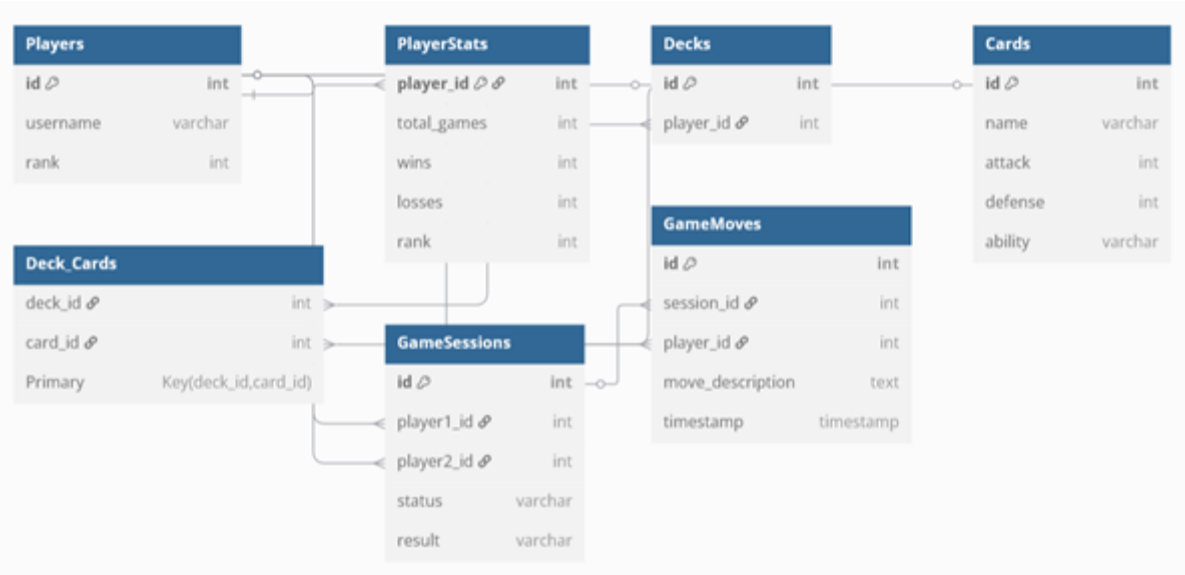
· **Decks** (id, player_id, card_list)

· **Cards** (id, name, attack, defense, ability)

· **Game Sessions** (id, player1_id, player2_id, status, result)

Database Model Diagram

A visual representation of the database schema has been included below.



Data Persistence

- **JSON Storage:** Used for temporary session data before committing to the database.
- **PostgreSQL:** Stores permanent game data such as user stats and game history.

Mathematical Model

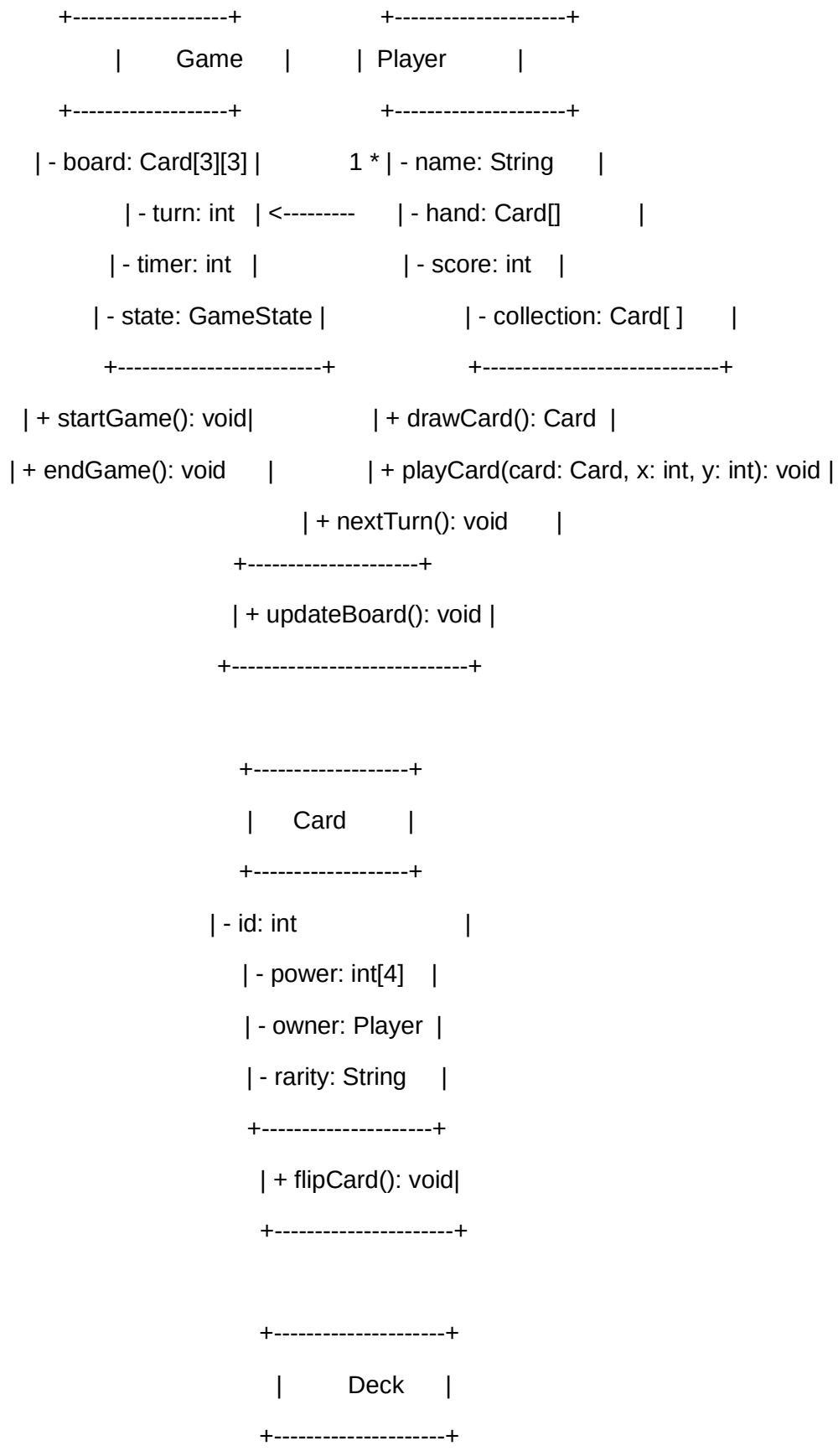
Mathematical models optimize game mechanics and data management. The project uses:

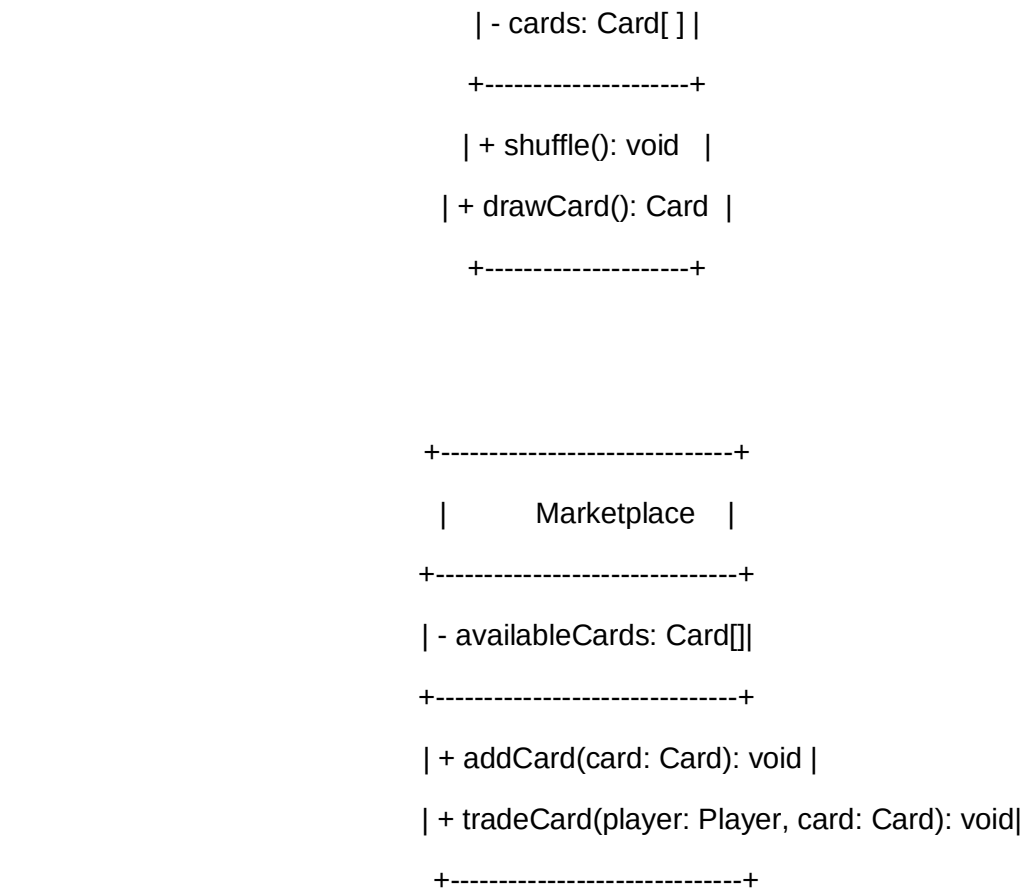
- **Hashmaps** for efficient lookups in deck management.
- **Array Operations** to update card effects dynamically.
- **Probability Calculations** to determine card draw probabilities.

Section 6: Class Diagram and Interface Specifications

Class Diagram

The Class Diagram provides a visual representation of the objects (classes), their relationships, and the key attributes and methods. Below is a textual representation of what the class diagram for Triple Tactics might look like:


Game
- board: Card[3][3]
- turn: int
- timer: int
- state: GameState
+ startGame(): void
+ endGame(): void
Player
- name: String
- hand: Card[]
- score: int
- collection: Card[ ]
+ drawCard(): Card
+ playCard(card: Card, x: int, y: int): void
+ nextTurn(): void
1 *
+ updateBoard(): void
Card
- id: int
- power: int[4]
- owner: Player
- rarity: String
+ flipCard(): void
Deck



Explanation of Class Diagram

- Game Class: Responsible for managing the overall game state. It handles the game board, keeps track of turns, and manages transitions (like moving to the next turn and ending the game).
- Player Class: Represents a player in the game. A player has a name, a hand of cards, a score, and a collection of cards they have collected.

- Card Class: Represents individual cards in the game. It holds information like the card's ID, power values on each side, its owner, and rarity. Cards can be flipped based on game mechanics.
- Deck Class: Represents the deck from which cards are drawn. It can shuffle and draw cards.
- Marketplace Class: Allows players to buy, sell, and trade cards. It tracks the cards available for trade and facilitates the exchange of cards between players.

Traceability Matrix (3) - Domain Concept Objects to Class Objects

Class	Requirement Traceability	Method(s)
CardGameRule (planned but unused)	Manages game rule sets and allows for alternate rulesets and game flows by modifying game states.	`name()`, `descrition()`, `icon()`
Playerdata	Manages individual player data, ip address, stats(unused), hand(unused currently was intended to be an anti-cheating measure)	`name()`, `id()`, `hand()`
Card	Represents a single card's attributes and interactions.	`getimage()` `get_description()` `get_border()` `get_title()` `total_value()` `clone()` `get_ai_score()`
Deck (unused, code was somewhat built but without help I was unable to implement the deckbuilder)	Manages the pool of cards used in gameplay.	`shuffle()`, `drawCard()`

Design Patterns

The Triple Tactics game uses several common design patterns to ensure flexibility, reusability, and maintainability:

- Singleton Pattern (Game): Ensures that only one instance of the Game class exists and is used throughout the game.

- Factory Pattern (Card Creation): Creates cards dynamically using a factory class to manage card generation, especially for different card types (rare, common, etc.).

- Observer Pattern (Game Updates): Players (observers) can be notified when the game state changes (e.g., end of the game, turn change).

- Strategy Pattern (Card Play Strategy): Allows different strategies for placing cards (e.g., offensive, defensive) based on player choice or game state.

Object Constraints

Object constraints are rules that govern the integrity of the data:

- Card Ownership: A card can only have one owner at a time, and the ownership can only change when it is placed on the board and outmatches an opposing card.

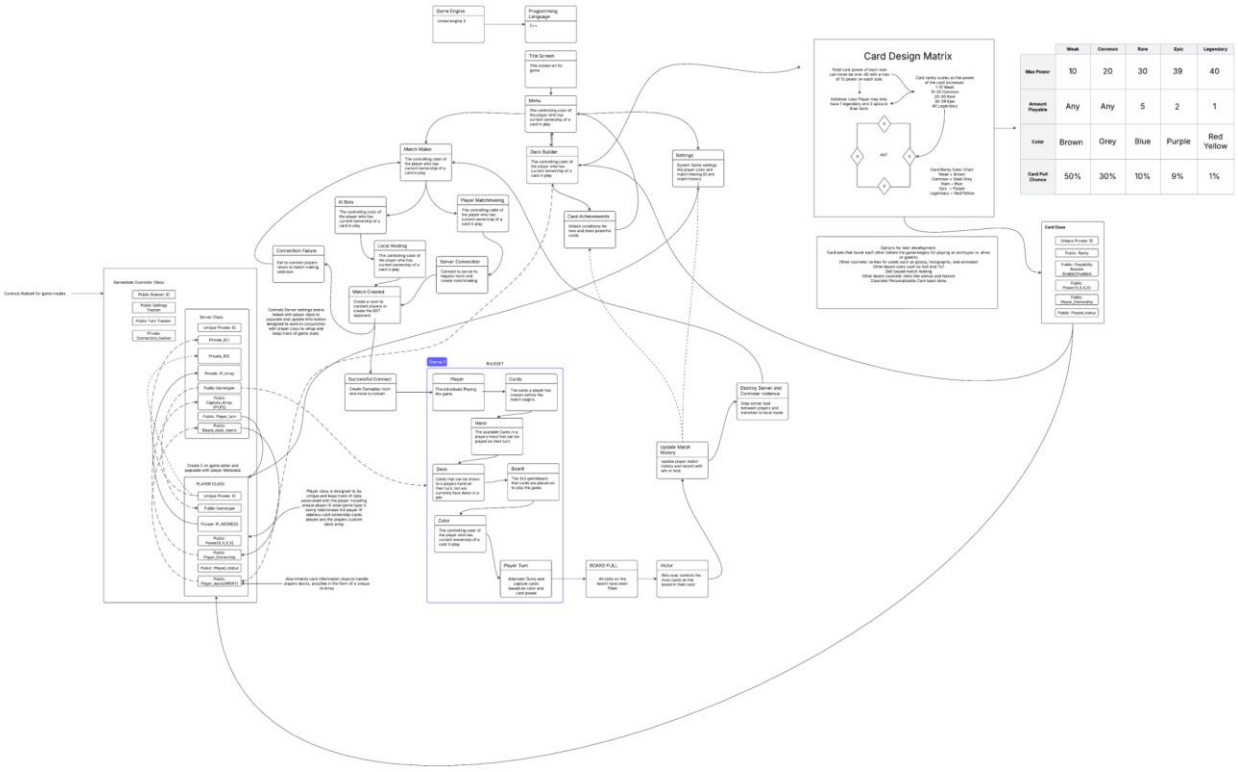
- Deck Size: The deck must always contain a certain number of cards. Players can only draw cards from the deck if there are cards remaining.

- Grid Boundaries: Players can only place cards in empty slots on the 3x3 grid. Once a card is placed, it cannot be moved.

- Game End Condition: The game ends when all slots in the grid are filled or when the board has no legal moves left. The player with the most cards on the board wins.

Section 7: Interaction Diagram

Total game interaction diagram



	Week	Common	Rare	Epic	Legendary
Max Power	10	20	30	39	40
Amount Possible	Any	Any	5	2	1
Color	Brown	Grey	Blue	Purple	Red Yellow
Card Pull Chance	50%	30%	10%	9%	1%

This interaction diagram is an extremely high level deep look at the communication and layout for the interactions between all planned gameplay elements. The above interaction chart displays what should interact between each element and how to design each modular element for the total program as a whole.

Objects and Design Principles:

The objects above are designed using Single responsibility and modular principles. This means that each individual application can both be single purpose but also be modified if needed and expanded to create modular objects inheriting their parent class.

Systematic Dependency:

This principle is applicable here as each object has some level of dependency from the objects above it as each parent class allows information to be passed on and inherited before being adapted for use. This can lead to faster execution as the information does not need to be created and instead is readily available from the parent object.

Section 8: System Architecture and System Design

Identifying Subsystems

- **Game Engine: Responsible for all the gameplay mechanics, including card placement, rule enforcement, power comparisons, win conditions, and game state management.**

- **User Interface (UI): Visual layout and interaction with the user, including the game board, cards, animations, and in-game feedback.**

- **Multiplayer: Manages the remote matches between players, including matchmaking, latency management, and synchronization.**

- **Card Collection: Handles the management of player card collections, including earning new cards, displaying collected cards, trading cards, and managing the economy around packs and rare cards.**

- **Matchmaking and Leaderboard: Manages matchmaking, ensuring that players are paired with opponents of similar skill levels, and tracks player progress on leaderboards.**

Architecture Style

- **Client-Server: The game's core logic runs on a centralized server, while the client-side (mobile devices) is responsible for rendering the game board, managing the**

user interface, and sending player actions to the server. This approach ensures that game data is stored securely on the server, and updates or changes to the game can be pushed centrally.

Angel Escalera implemented a Flask backend for this purpose, which communicates with Ren'Py's front-end using HTTP requests to send game state data, player actions, and updates.

- **Microservices-based Backend: The backend consists of independent services, each focusing on specific tasks (e.g., card collection management, multiplayer services, player authentication, etc.). This approach allows for better scalability, maintainability, and independent updates to each subsystem without affecting the entire system.**

  Angel Escalera's backend uses Flask, where RESTful APIs manage core gameplay features like player registration, session tracking, and game state updates.

- **Event-Driven: The interaction between the client and the server can be event-driven, allowing the system to react to player actions (e.g., placing a card, flipping a card, completing a match) efficiently and in real-time.**

  Through the Flask API developed by Angel Escalera, the server handles requests from the client (Ren'Py) to process player actions and synchronize game state.

Mapping Subsystems to Hardware

- **Hardware: Runs on a mobile device (smartphone or tablet). It requires a CPU and GPU to handle the real-time processing of game logic and graphics rendering.**

- **Scaling: Distributed across the game client and servers (in the case of multiplayer games).**

- **User Interface:**

    - **Hardware: Mobile device screen, touchscreen for interaction, and audio systems for sound effects and voice feedback.**

- ***Ren'Py handles the visual representation and user input, while the server tracks the game state and player interactions, with the server functionality built by Angel Escalera.***

External Connections and Network Protocols

- **Game Servers: The client communicates with game servers for matchmaking, game state synchronization, and leaderboard updates.**

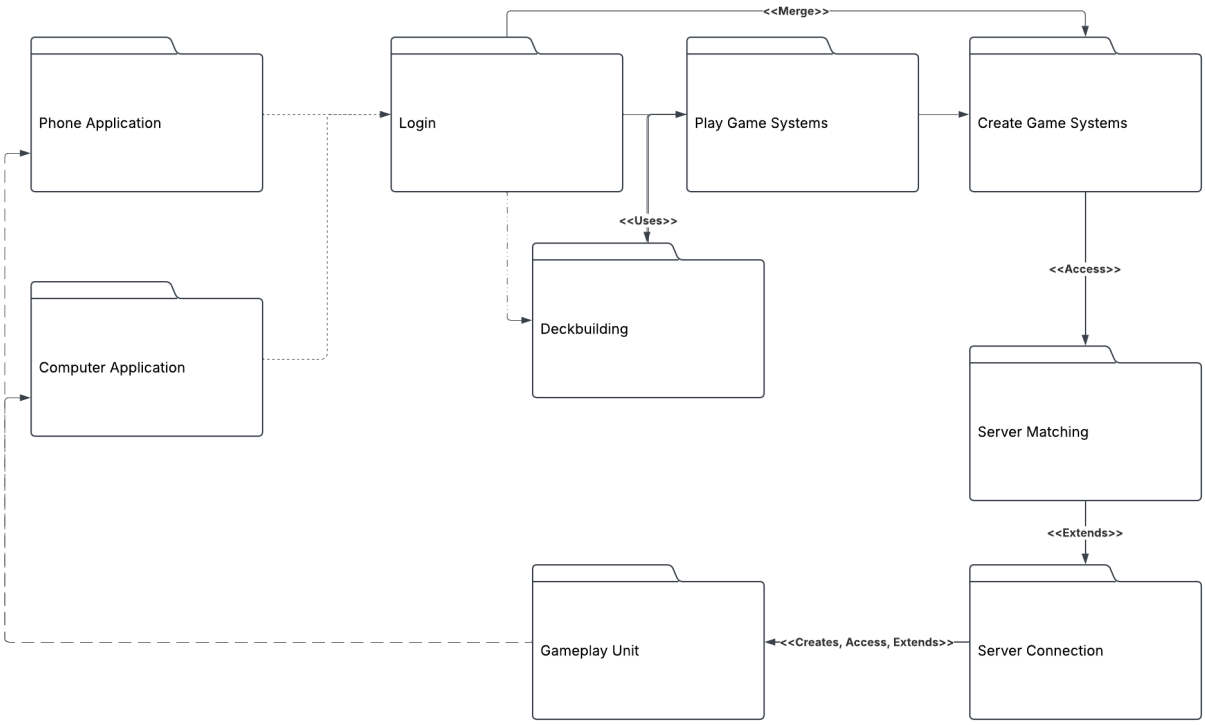
The Flask server built by Angel Escalera serves as the central hub for multiplayer matches, managing real-time data, including player actions, turn order, and game outcomes.

- **Global Control Flow:**
    - **HTTP/HTTPS: Used for communication between the client and the server, especially for non-real-time interactions like profile updates.**
    - **TCP/IP: Ensures reliable, ordered data transmission between servers and clients, which is crucial for multiplayer synchronization.**
- ***This allows the client (Ren'Py) to communicate with the server efficiently for all game actions and data synchronization, as designed by Angel Escalera.***

General Hardware Requirements

- **Minimum Requirements: Android or iOS device with at least 2GB RAM and a modern GPU (for smooth graphics rendering).**
- **Recommended: Devices with 4GB RAM or higher, with a better GPU for smooth performance on high-end devices.**

Identifying Subsystems via UML Chart


<<Merge>>
Phone Application
Login
Play Game Systems
Create Game Systems
<<Uses>>
<<Access>>
Deckbuilding
Computer Application
Server Matching
<<Extends>>
Gameplay Unit
<<Creates, Access, Extends>>
Server Connection

Section 9: Algorithms and Data Structures

Algorithms

The algorithms used in **Triple Tactics** play a critical role in managing game state, determining win conditions, shuffling cards, and handling multiplayer communication. Here are some of the key algorithms used:

1. **Shuffling Algorithm (Deck)**:

    - **Purpose**: Shuffle the deck to randomize the order of the cards before the game starts. A standard shuffling algorithm used is **Fisher-Yates (Knuth shuffle)**, which ensures that each card has an equal chance of appearing at any position in the deck.
    - **Algorithm**:
        - Iterate through the deck from the last card to the first.
        - Swap the current card with a randomly chosen card from earlier in the deck.

JAVA:

```
public void shuffle() {

    Random rand = new Random();

    for (int i = deck.size() - 1; i > 0; i--) {

    int j = rand.nextInt(i + 1);

    Collections.swap(deck, i, j);

    }

}
```

2. **Turn Handling Algorithm**:

- **Purpose**: Manage the flow of turns between players, ensuring that players can only take their turn when it's appropriate and that the game progresses smoothly.
- **Algorithm**:
  Track the current turn number.
    - After each turn, update the game state and switch to the next player

JAVA:

```
public void nextTurn() {

    turn++;

    // Check if all slots are filled or win condition is met

    if (isGameOver()) {

    endGame();

    }

}
```

3. Card Placement and Interaction:

- **Purpose**: Determine if a card can be placed in a given slot and whether it will flip any opposing cards. Cards interact by comparing their power values based on their sides (top, bottom, left, right).
- **Algorithm**:
    - When a card is played, check its neighboring cards on the board.
    - Compare the card’s power to the neighboring card's corresponding side.
    - Flip the opponent’s card if the player’s card has a higher power.

JAVA:

```
public boolean canPlaceCard(Card card, int x, int y) {

    if (board[x][y] == null) {
```

```
        return true; // slot is empty
        }
        return false;
}

public void flipCard(Card card, int x, int y) {
        if (card.power[1] > board[x][y].power[3]) { // Example: Right side vs Left side
        board[x][y] = card;
        }
}
```

4. Win Condition Algorithm:

- Purpose: Determine if the game has ended and if a player has won. The game ends when all board positions are filled or when one player has the most cards.
- Algorithm:
    - Check if all positions on the board are filled.
    - Count the number of cards owned by each player.
    - The player with the most cards on the board wins.

JAVA:

```
public boolean isGameOver() {
        int filledSlots = 0;
        for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
```

```
            if (board[i][j] != null) {
                    filledSlots++;
            }
            }
            }
            return filledSlots == 9;
}

public void endGame() {
            // Calculate scores
            int playerOneScore = countCards(Player.ONE);
            int playerTwoScore = countCards(Player.TWO);
            if (playerOneScore > playerTwoScore) {
            System.out.println("Player One wins!");
            } else {
            System.out.println("Player Two wins!");
            }
}
```

Data Structures

In Triple Tactics, several data structures are crucial for efficiently managing the game's flow, card mechanics, player data, and multiplayer functionality. Below are the primary data structures used and their roles:

1. **Array (2D Array for the Board):**

- Purpose: The 3x3 game board is represented using a 2D array, where each cell contains a `Card` object. This allows for quick access to each slot on the board and facilitates the placing and flipping of cards.
- Structure: A 2D array `Card[3][3]`, each element representing one of the nine slots on the board.

  JAVA:
- Card[ ][ ] board = new Card[3][3];

2. **ArrayList (for Hand and Collection):**

- Purpose: Player's hand and card collection are represented using an `ArrayList<Card>`. This allows for dynamic resizing as players draw or collect cards.
- Structure: The `ArrayList` is chosen because it supports efficient appending and removing of cards, which is required when players draw cards, play them, or trade them.

JAVA:

```
ArrayList<Card> hand = new ArrayList<Card>();
ArrayList<Card> collection = new ArrayList<Card>();
```

3. **LinkedList (for Deck):**

- **Purpose**: The deck, which contains all the cards that have not yet been drawn, is represented as a `LinkedList<Card>`. This structure is useful as it supports efficient insertions and deletions, particularly when shuffling the deck or drawing cards.
- **Structure**: A `LinkedList` provides O(1) time complexity for adding/removing cards from either end, making it ideal for shuffling and drawing.

JAVA:

```
LinkedList<Card> deck = new LinkedList<Card>();
```

4. HashMap (for Marketplace):

- Purpose: The marketplace, where players trade cards, can be efficiently managed using a `HashMap<Integer, Card>`, where the key is the card ID and the value is the `Card` object. This allows quick lookups, insertions, and deletions of cards.
- Structure: The `HashMap` helps in checking whether a card is available in the marketplace and quickly retrieving card details when a trade happens.

JAVA:

```
HashMap<Integer, Card> marketplace = new HashMap<Integer, Card>();
```

Concurrency

In a multiplayer setting, Triple Tactics will need to handle concurrent actions, such as two players playing at the same time or multiple actions happening simultaneously (e.g., drawing cards and updating the game board). Here's how concurrency can be addressed:

1. Threading for Multiplayer:

- Purpose: To allow two players to play the game at the same time, we can use threads to manage each player's actions without blocking the other.
- Approach: Each player can be assigned to a separate thread. The game will switch between threads for each player's turn.

JAVA:

```
class PlayerThread extends Thread {

    private Player player;
```

```
    private Game game;


    public PlayerThread(Player player, Game game) {
    this.player = player;
    this.game = game;
    }


    @Override
    public void run() {
    while (!game.isGameOver()) {
    player.playTurn();
    game.nextTurn();
    }
    }
}
```

2. Synchronized Methods:

- Purpose: Ensure that no two threads access shared resources simultaneously, which can lead to race conditions (e.g., updating the board or trading cards).
- Approach: Use synchronized methods to ensure that only one thread at a time can update critical sections of the code (e.g., updating player scores or card positions on the board).

JAVA:

```
public synchronized void updateBoard(Card card, int x, int y) {
        if (board[x][y] == null) {
        board[x][y] = card;
        }
}
```

3. Atomic Operations for Card Drawing and Trading:

- Purpose: Use atomic operations to ensure that card drawing and trading operations are done safely in a concurrent environment.
- Approach: Use atomic classes or locks to manage critical operations, ensuring that one player can’t draw a card while the other player is drawing.

JAVA:

```
class AtomicCardDrawer {
        private final Lock lock = new ReentrantLock();


        public Card drawCard(Deck deck) {
        lock.lock();
        try {
        return deck.drawCard();
        } finally {
        lock.unlock();
```

```
        }

        }

}
```

Conclusion

The combination of well-chosen data structures (arrays, linked lists, hash maps, etc.) and efficient algorithms (shuffling, turn management, card interactions) ensures that Triple Tactics runs smoothly and efficiently. Concurrency is handled through threading and synchronization techniques to maintain a seamless multiplayer experience. These strategies help maintain high performance while ensuring a rich, interactive experience for players.

Section 10: User Interface Design and Implementation

User Interface Design:

- Minimalist Design: The UI should be clean, showing only necessary elements—game board, player stats, card details, and the timer.
- Card Display: Cards in the player's hand should be displayed in a scrollable area at the bottom of the screen, with each card showing its power values.
- Game Board: The 3x3 grid should be clearly visible, with slots that update in real time to show the current state of play.
- Indicators: Player turns, remaining time, and player scores should be visible throughout the game.
- Animations: When a card is placed or flipped, there should be animations to provide clear feedback to the player. This includes flipping cards and showing a winning player at the end.

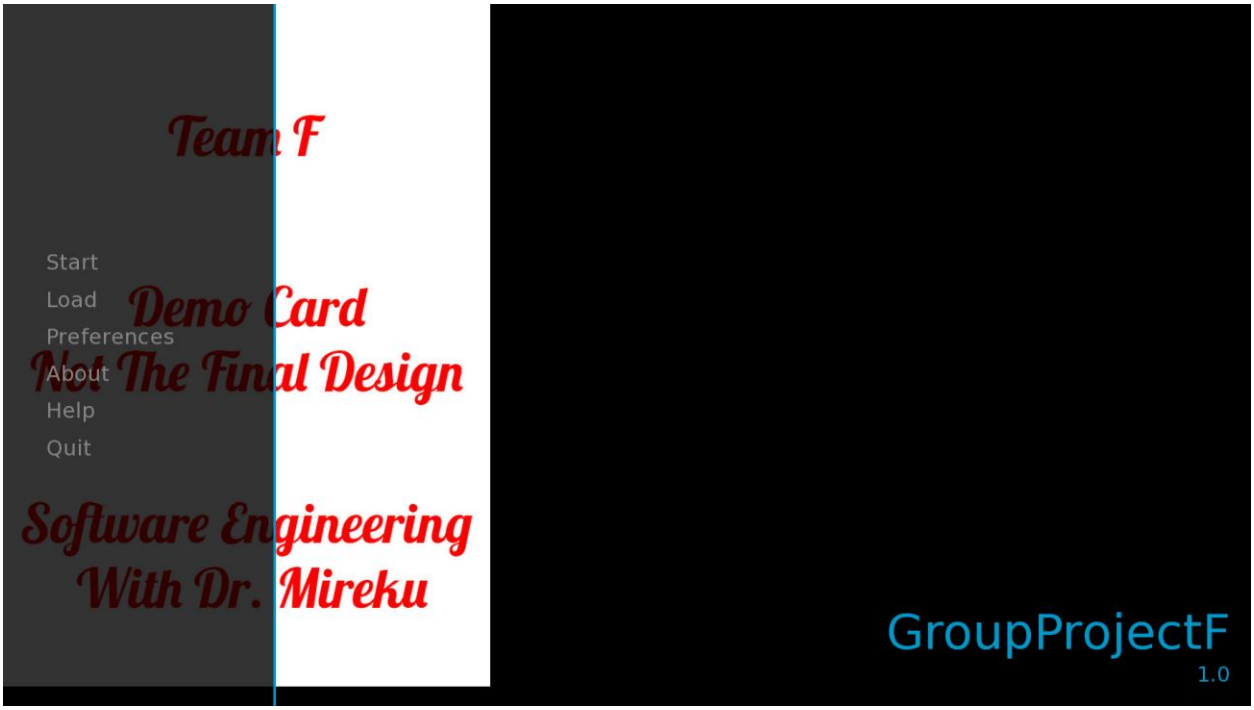
Interaction Diagram:

- This is a publisher subscriber model.
- The card game assets are the board and the cards themselves.
- These cards both come in blue and red, having two versions of the same animal with the same name each.

- There is an animal such as Milo the husky which has two versions of him and two colors of him which equals 4 cards of Milo.
- Triple triad consists of different types of designs and the design chosen for this game specifically was animals, hence the above comments.
- To complete the cards, Photoshop was used and the animals were looked up online. PNG images of the animals were downloaded and added into a card also designed from scratch.
- The cards consist of the colors blue and red and have a texture behind them.
- The little designs on the upper cards are paint brushes from Photoshop.

Section 10a: Implemented UI Design

Main Menu look, if we were headed to market this would be replaced with actual game art and assets.



Game board UI

A clear and easy to understand game board, the game board is split into a 3x3 grid that cards can be placed on.

Card Battle Matrix

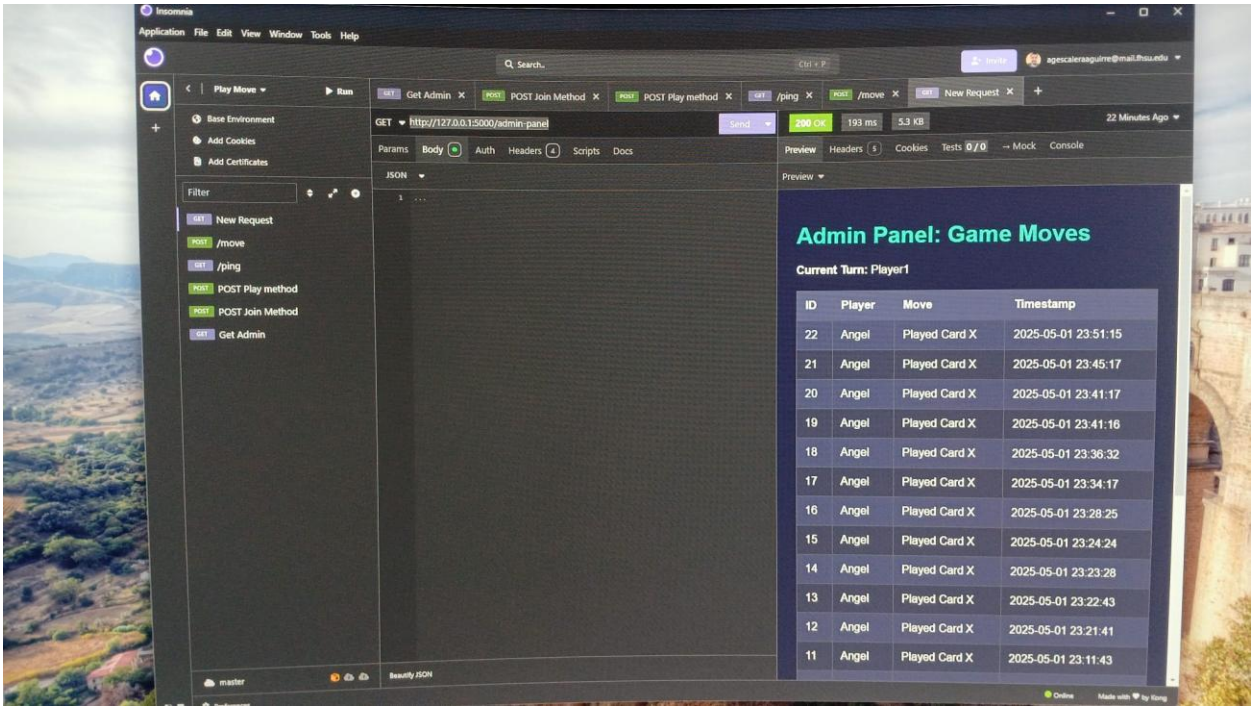
Player 1 places card 1 down on the table indicated by the red card on the board

Player 2 then places their own card down on the table as show by the card in blue.

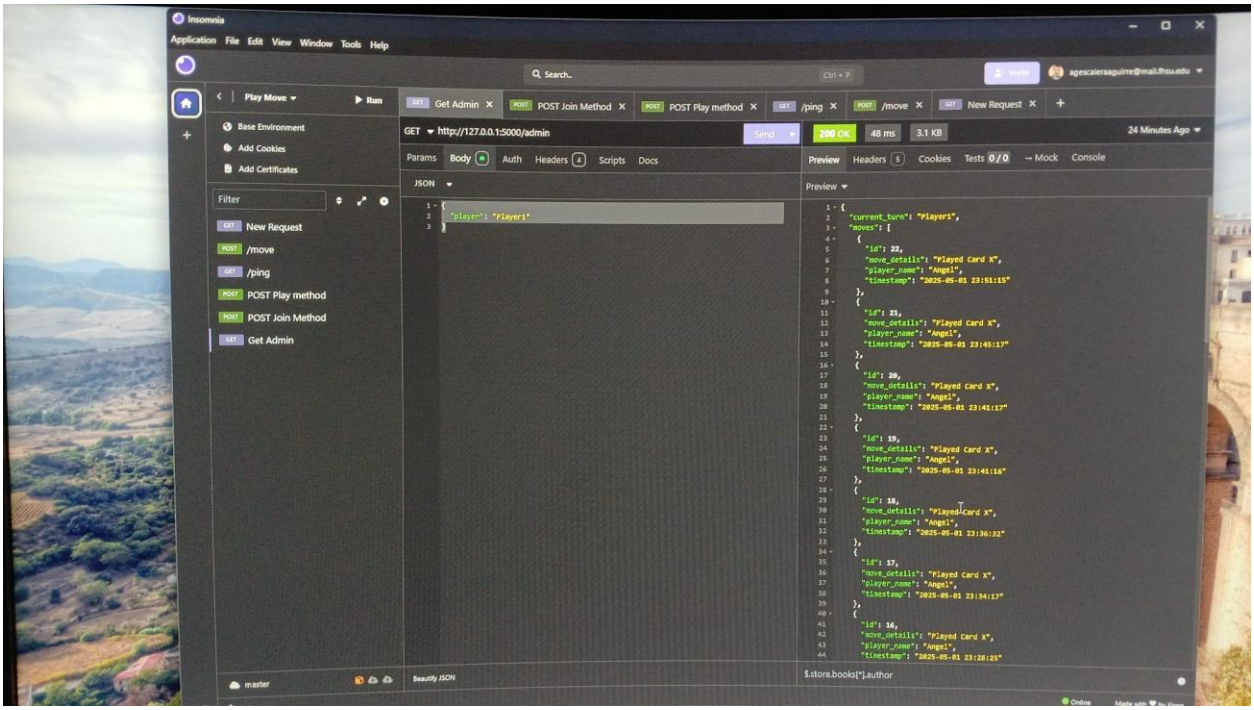
Second Card is Placed down and card 1 ownership is taken by player 2

Section 10B: Server side UI/GUI

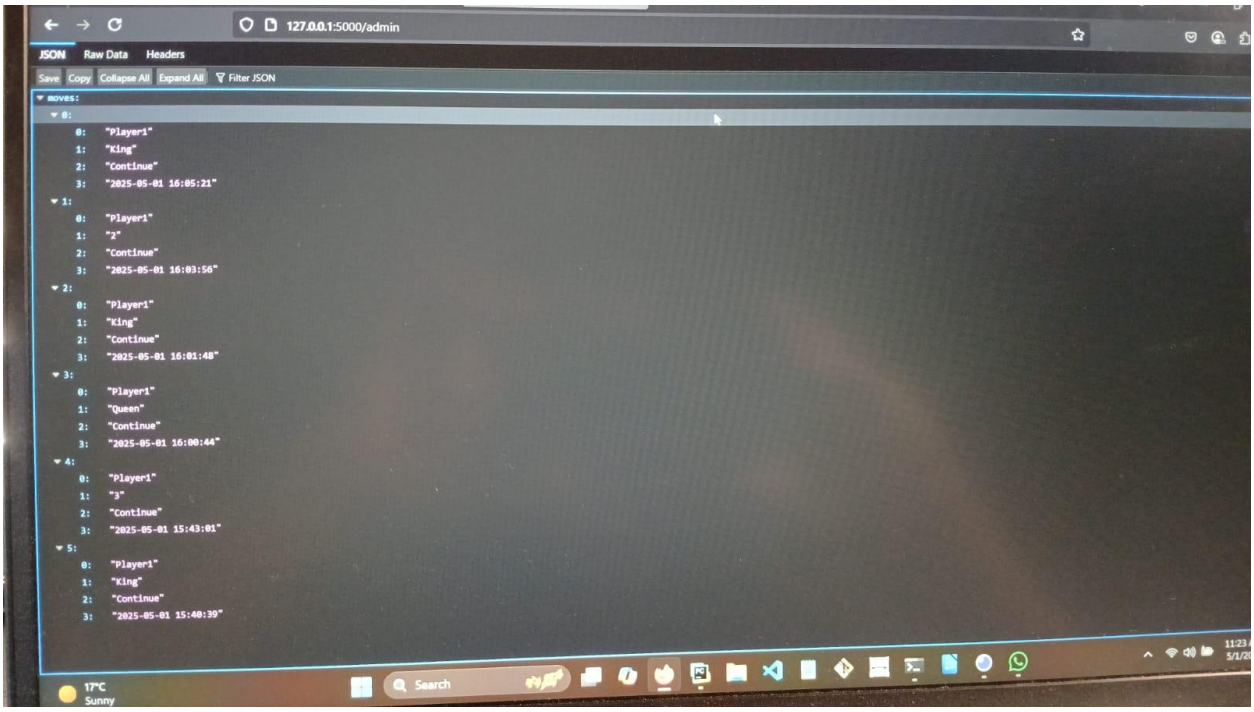
Implementation of server API in developer view



Raw Server JS file tracking moves from each card



Administrative server Json showing move history within server browser for showing locality



Section 11: Design of Tests

Objective: Ensure that the player can place cards correctly on the 3x3 grid.

Preconditions: The game has started, and the player has cards in their hand.

Test Steps:

1. Select a card from the player's hand.
2. Drag and drop the selected card onto an empty slot on the grid.
3. Confirm placement.

Expected Results:

- The card should be placed on the grid at the selected position.
- The game board should update to reflect the new card placement.

Pass Criteria: The card is placed successfully, and the game state is updated.

Fail Criteria: The card cannot be placed, or the board does not update.

Timer Countdown

Objective: Validate the timer functionality to ensure the match lasts no longer than 45 seconds.

Preconditions: The game must be in progress.

Test Steps:

1. Start a new match.

2. Observe the countdown timer.

Expected Results:

- The timer should count down from 45 seconds.
- The game should end when the timer reaches zero.

Pass Criteria: The timer runs down to zero, and the game ends.

Fail Criteria: Timer does not count correctly, or the game continues after time runs out.

Objective: Ensure that players can earn new cards after winning a match.

Preconditions: Player has completed at least one match.

Test Steps:

1. Complete a match and win.
2. Check the player's card collection.

Expected Results:

- A new card should be added to the collection.
- The player receives a notification for the new card.

Pass Criteria: The new card is added to the player's collection.

Fail Criteria: No card is added to the collection, or no notification is shown.

Multiplayer Matchmaking

Objective: Verify the matchmaking process works correctly and players are matched based on their skill level.

Preconditions: Two players must be available for matchmaking.

Test Steps:

1. Both players initiate a matchmaking request.
2. Wait for the system to match them.

Expected Results:

- The system should find a match for the players within a reasonable time.
- The match should be fair based on player skill level (if applicable).

Pass Criteria: Players are matched, and the game starts with fair conditions.

Fail Criteria: Players are not matched in a reasonable time, or the matchmaking doesn't consider skill levels.

Game Load Time

Objective: Ensure that the game loads quickly.

Preconditions: User is opening the app.

Test Steps:

1. Launch the game on a device.

2. Measure the time taken for the game to be fully loaded.

Expected Results:

- The game should load within 5 seconds.

Pass Criteria: The game loads within 5 seconds.

Fail Criteria: The game takes longer than 5 seconds to load.

Network Latency in Multiplayer

Objective: Verify that remote multiplayer matches have minimal latency.

Preconditions: Both players must be connected to the internet.

Test Steps:

1. Start a multiplayer match.

2. Measure the delay between the players' actions.

Expected Results:

- There should be minimal latency in actions, such as card placement and grid updates.

Pass Criteria: Latency is under 200 ms.

Fail Criteria: Latency is over 200 ms, leading to a noticeable delay in actions.

Stress Test (Multiple Simultaneous Players)

Objective: Ensure the game can handle multiple users simultaneously without crashing or performance degradation.

Preconditions: Game servers are up and running.

Test Steps:

1. Simulate a high volume of concurrent players (e.g., 1,000 players).
2. Measure game performance and server stability.

Expected Results:

- The game should maintain performance with no significant lag or crashes.
- Players should be able to start and finish games without interruption.

Pass Criteria: No performance degradation, and the system can handle multiple users.

Fail Criteria: Significant lag, crashes, or inability to connect.

Server Integration and Backend Development

As part of the card game project for Software Engineering, **Angel Escalera** was responsible for designing and implementing the backend server using Flask and integrating it with the front-end built in Ren'Py. The goal was to enable real-time, multi-player game logic, persistent data storage, and game-state communication between players through a central server.

Design Overview

The backend was built using Python's Flask framework, which was chosen for its simplicity and compatibility with Ren'Py. **Angel Escalera** implemented a RESTful API structure to support core gameplay features, including:

- Player registration and session tracking
- Turn-based multiplayer interactions
- Deck and hand management
- Move logging and game history tracking
- Win/loss detection
- Admin panel with game data insights

SQLite was also used for persistent storage, allowing the backend to store detailed data about each move, player actions, and match outcomes. This data could then be queried or displayed through an admin interface. The database schema included tables for players, cards in play, and move history.

Integration with Ren'Py

Ren'Py, being a visual novel engine, required careful design to interface with the web backend. **Angel Escalera** used Python's requests module inside Ren'Py's script blocks to send HTTP requests to the Flask server. This allowed Ren'Py to:

- Fetch game state data from the server
- Send player actions (like playing a card or ending a turn)
- Update the UI based on the server's response

Ren'Py's screen system was customized to build an interactive card table, using the existing GUI screens and adapting the styles to match the game's aesthetic. This included modal popups, win/loss messages, and player input screens.

Challenges Faced

Integrating a real-time multiplayer server with a visual novel engine like Ren'Py presented several unique challenges:

- **Limited Native Support for Networking:** Ren'Py doesn't natively support multiplayer or socket communication. To work around this, **Angel Escalera** implemented REST APIs on the server and used synchronous HTTP requests in Ren'Py to simulate real-time interaction.

- **Turn Management:** Ensuring that only the correct player could act during their turn required careful state management on both the server and client. **Angel Escalera** implemented server-side turn validation and client-side turn indicators.

- **UI Complexity in Ren'Py:** Ren'Py's screen language and ATL (Animation and Transformation Language) are not optimized for dynamic game states like a card hand or board layout. **Angel Escalera** had to use creative workarounds with screen variables and custom styles to dynamically update the UI based on server responses.

- **Data Consistency and Error Handling:** Keeping the front-end state in sync with the server required reliable error checking and retry mechanisms in Ren'Py. **Angel Escalera** built in confirmation messages and server response validation to ensure a consistent user experience.

- **Deployment and Testing:** To facilitate testing, **Angel Escalera** hosted the Flask server locally during development. A version of the backend was also created that can be easily hosted on platforms like PythonAnywhere or Heroku for future deployment needs.

Features Implemented

- Full API documentation for team use
- SQLite integration with schema migrations
- Admin panel for move and match history
- Game win/loss detection with UI feedback
- Front-end screens for player interaction, confirmation, and notifications
- Multiplayer support with backend state tracking

Section 12: Project Management and Plan of Work

Outlining format report

Team leader Iain Fehr outlined and formatted the report using one of the example projects as a guide and used the changelog as a way to document current changes to the document going forward, going forward all work should and would be completed on the document to ensure that each member documents and commits to their assigned tasks.

Project Coordination

The current project communication is limited by whatsapp communication but verified by Teamleader Iain Fehr with read receipts to ensure that each member is briefed on the goals and situations going forward after the meeting with the professor. Team members were assigned specific goals and encouraged to collaborate as necessary when specific topics were assigned to multiple people. Project individuals should also document work done within the change log posted.

History of the Report and Changelog

Change Log:

- 3/12
    - Iain -Met with Professor and received feedback for project management and assignment going forward to total group
    - Iain- Communicated with group new direction and changes to the project that needed to be made
- 3/14
    - Iain- Assigned goals and requirements to each individual in the group for new direction going forward
- 3/17
    - Iain- Reformatted report and broke down the project into proper sections
    - Iain - Section 1 completed and formatted
    - Iain- Reported over use cases, needs to be further refined
- 3/20
    - Iain- communicated the Urgency of the current status of the project
    - 3/21
    - Iain- Reformatted Report again for usable breakdown and readability and continuing development of Use cases and User Interface Development. Updated references, glossary and project management pages
- 3/23
    - Juan - Added nonfunctional requirements to the document such as Performance, Security, Usability, Reliability, Maintenance and Compatibility.
    - Juan - Added the architecture style such as System Architecture and Design, Identifying Subsystems, Architecture Style, Subsystems and Hardware, External Connections and Network Protocols, Global Control Flow and General Hardware Requirements.
    - Juan - Added traceability matrix
    - Seth- Formatted document and added information personally responsible for. looked over assignment before submission
    - Angel- Created and added proposed changes to the project statement, added requirements looked over the assignment before submission and gave personal approval

    - Iain- Completed Section 6, reformatted Section 4, completed the system interaction Section 7 updated project management, moved changelog to project management section, and updated references section. Formatted as much of the document as possible. Removed Duplicate section 6 and updated section labels to match accurately. Will submit Assignment imported UML chart from previous documents in section 8
    - Seth Tharo - Finalized all of his parts and formatted the layout of the whole document.
- 3/20
    - Iain- Issues with unreal engines ability to be shared among group members encountered game engine switched to python based Renpy to fix issue.
    - Little communication from Seth and Sinan forces reassignment of availible group memebers, Juan busy at time Angel tasked with Backend Development of server, Iain self reassigned from asset creation to UI and gameplay development.
- 4/5
    - Iain- First instance of the game code booting up without runtime issues
    - Angel - Continues Work on Server side application
    - Juan tasked to game assets.
- 4/10
    - Iain- Server side code booted up on local environment outside Angels computer
    - Last known communication with Seth Tharo
- 4/11
    - Iain- Received Game assets from Jaun, implemented then recorded and submitted Group project.
- 5/3
    - Iain- Implemented and modified parts of document to reflect changes in game engine
- 5/4
    - Juan - Added references for card game assets and updated the interaction diagram on how the cards are, how they were designed and how it was done.

Plan of Work

	Febuary				March				April				May		
	1-8	8-15	15-22	22-28	1-7	8-15	16-22	23-29	30-4	5-11	12-19	20-25	26-2	2-9	10
Iain Fehr	Learn Unreal Engine					Create 2D Game enviroment		Create Base Card System		Game Art and Front Design					
Angel Escalera	Learn Unreal Engine						Create Title Menu		Create Deckbulding system		Card Creation	Gameplay Testing	Local BOT Systems	Final QA Tests	TURN IN
Juan Enriquez	Learn Unreal Engine						Create Title Menu		Client Matchmaking REST		Gameplay Testing	Card Creation	Local BOT Systems	Final QA Tests	TURN IN
Seth Tharo	Learn Unreal Engine						Create Base Card System		Create Deckbuilding System		Gameplay Testing	Card Creation	Local BOT Systems	Final QA Tests	TURN IN
Sinan	Learn Unreal Engine						Create REST API for server		Client Matchmaking REST		Card Creation	Gameplay Testing	Local BOT Systems	Final QA Tests	TURN IN

The above was a prototype implementation of the timeline long since replaced after feedback was received,Instead the timeline is now based on the work Assignment located at the top of the assignment in addition to a general timeline.

Estimated Timeline

- 3/23
    - Finish report 2 and submit the full report, then start development of Demo 1 using Report 2 as project implementation guide
- 4/11
    - Submit Demo #1 Ensuring that the local and basic web app is created to specification, Start refining process for for Demo 2 using feedback for further development and refinement of prototype

- 4/15
    - Small group break to catch up on other classes
- 4/16
    - Continue Refining Demo and modifying report 2 into report 3 using information gathered from the project.

- 5/1
    - Complete Demo 2 and use remaining 5 days for bug testing and updating of Report 2 to update it to Report 3

- 5/4
    - Finish updating Report 2 to Report 3 using information from development and submit the completed report.
- 5/5

    - Continue Development of Game bug testing and capabilities for launch until 5/12 when final submission of project is due.
- 5/12
    - Create and Submit the Video report and demo of the project

Section 13: References

Repository of Project: https://github.com/icfehr/Team_F_Project.git

1. **Fisher-Yates Shuffle Algorithm**
    - **Title**: Knuth Shuffle (Fisher-Yates Shuffle)
    - **Source**: Knuth, D. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
    - **Description**: Provides an explanation and implementation of the Fisher-Yates shuffle algorithm, which is commonly used for shuffling cards in card games.
2. **Java Concurrency in Practice**
    - **Title**: Java Concurrency in Practice
    - **Author**: Brian Goetz
    - **Publisher**: Addison-Wesley Professional (2006)
    - **Description**: A comprehensive guide to handling concurrency in Java, offering best practices, pitfalls, and design patterns to handle concurrent programming, which is essential for multiplayer functionality in Triple Tactics.
3. **Data Structures and Algorithms in Java**
    - **Title**: Data Structures and Algorithms in Java
    - **Author**: Robert Lafore
    - **Publisher**: Sams Publishing (2002)

- **Description**: Provides in-depth coverage of fundamental data structures and algorithms, with Java implementations. Useful for understanding data structures like arrays, linked lists, and hash maps used in Triple Tactics.

4. **Game Design Patterns**
    - **Title**: Game Programming Patterns
    - **Author**: Robert Nystrom
    - **Source**: *gameprogrammingpatterns.com*
    - **Description**: A collection of design patterns and techniques used in game development, focusing on performance, code reuse, and maintainability. Particularly useful for the design of game logic and systems like those used in Triple Tactics.
5. **Java Documentation**
    - **Title**: Official Java API Documentation
    - **Source**: Oracle Corporation. *Java SE Documentation*
    - **URL**: https://docs.oracle.com/en/java/javase/
    - **Description**: The official Java API documentation is a reference for understanding the Java standard library and various methods and classes available for implementing the game mechanics and concurrency management.
6. **Triple Triad Card Game**
    - **Title**: Final Fantasy Triple Triad

- **Source**: Square Enix
- **URL**: https://finalfantasy.fandom.com/wiki/Triple_Triad
- **Description**: The official page for **Triple Triad**, the inspiration for **Triple Tactics**. It provides an overview of the game's mechanics, rules, and card properties.

7. **Design Patterns: Elements of Reusable Object-Oriented Software**
   - **Title**: Design Patterns: Elements of Reusable Object-Oriented Software
   - **Authors**: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides
   - **Publisher**: Addison-Wesley Professional (1994)
   - **Description**: A foundational work on object-oriented design patterns that helped shape modern software architecture, useful for structuring game systems like card collection, multiplayer matchmaking, and user interface design.
8. **Multiplayer Game Architecture**
   - **Title**: Multiplayer Game Programming: Architecting Networked Games
   - **Author**: Joshua R. P. H.
   - **Publisher**: New Riders Publishing (2003)
   - **Description**: This book provides techniques for designing networked multiplayer games, essential for building the multiplayer functionality in **Triple Tactics** with minimal lag.
9. **The Art of Game Design: A Book of Lenses**

- **Author**: Jesse Schell
- **Publisher**: CRC Press (2008)
- **Description**: Offers insights into game design principles, including mechanics, game flow, and the overall player experience, which were pivotal in shaping the gameplay and user experience for **Triple Tactics**.

10. **Agile Software Development**
    - **Title**: Agile Software Development, Principles, Patterns, and Practices
    - **Author**: Robert C. Martin
    - **Publisher**: Prentice Hall (2002)
    - **Description**: Provides a comprehensive understanding of Agile software development practices, which are vital for iterative development and project management, particularly in a collaborative team setting like **Triple Tactics**.
11. **Flash Card App Project**
    - **Title**: Flash Card App
    - **Source**: *Fort Hays State University - CSCI 441 Software Engineering Projects*
    - **URL**: https://www.fhsu.edu/cs/csci441/se/projects/sampleprojects/flash_card_app.pdf
    - **Description**: This document provides insights into the structure and implementation of a flash card app project, focusing on software engineering best practices, object-oriented design, and interface development. Useful for understanding how to approach mobile app development and user interface design for games like **Triple Tactics**.

12. **Game Assets**
    - **URLs for the animal pngs:**
    - https://www.google.com/url?sa=i&url=https%3A%2F%2Fpngtree.com%2Ffreepng%2Fsiberian-husky-dog_16179695.html&psig=AOvVaw0LABUV78ZFvuiv9L_NMc2a&ust=1746498136204000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCIj4x9mii40DFQAAAAAdAAAAABAE
    - https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.pixelsquid.com%2Fpng%2Fsitting-husky-dog-black-and-white-2695145005525243580&psig=AOvVaw0LABUV78ZFvuiv9L_NMc2a&ust=1746498136204000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCIj4x9mii40DFQAAAAAdAAAAABAI
    - https://png.pngtree.com/png-vector/20230318/ourmid/pngtree-duck-poultry-animal-transparent-on-white-background-png-image_6653170.png
    - https://www.google.com/url?sa=i&url=https%3A%2F%2Fpngimg.com%2Fimage%2F5011&psig=AOvVaw1kWmUgy0j8np6k3HzQxBBI&ust=1746498270171000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCNi1n6eji40DFQAAAAAdAAAAABAJ
    - https://www.google.com/url?sa=i&url=https%3A%2F%2Fclipartpng.com%2F%3F1046%2Cbrown-horse-png-clipart&psig=AOvVaw2mhULV_6RlVO8mYkxPqLkU&ust=1746498354278000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCKiEusGji40DFQAAAAAdAAAAABAE
    - https://www.google.com/url?sa=i&url=https%3A%2F%2Fpngimg.com%2Fimage%2F302&psig=AOvVaw2mhULV_6RlVO8mYkxPqLkU&ust=1746498354278000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCKiEusGji40DFQAAAAAdAAAAABAI
    - https://www.google.com/url?sa=i&url=https%3A%2F%2Fpngimg.com%2Fimage%2F24801&psig=AOvVaw112jq3tl9q38QlGAW_bmI-&ust=1746498392268000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCJCvn9Sji40DFQAAAAAdAAAAABAE
    - https://www.google.com/url?sa=i&url=https%3A%2F%2Fpngimg.com%2Fimage%2F24795&psig=AOvVaw112jq3tl9q38QlGAW_bmI-

&ust=1746498392268000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCJCvn9Sji40DFQAAAAAdAAAAABAJ

Work Cited:

ARR: Triple Triad. (n.d.-a). *ARR: Triple Triad - Final Fantasy XIV*. Retrieved May 4, 2025, from https://arrtripletriad.com/

FFXIV Collect. (n.d.). *Cards*. FFXIV Collect. Retrieved May 4, 2025, from https://ffxivcollect.com/triad/cards

References (APA Style)

Python Software Foundation. (2023). *pip - The Python Package Installer*. https://pip.pypa.io/

JetBrains. (2024). *PyCharm Documentation*. https://www.jetbrains.com/pycharm/documentation/

FastAPI. (2024). *FastAPI Documentation*. https://fastapi.tiangolo.com/

Pallets Projects. (2024). *Flask Documentation*. https://flask.palletsprojects.com/

SQLAlchemy. (2024). *SQLAlchemy Documentation*. https://docs.sqlalchemy.org/

Final Report – Installed Python Packages

Author: Angel Escalera

Project: Team F – Backend Development

Environment: PyCharm, Python Virtual Environment (`.venv`)

Date: [5/4/2025]

Summary

This report provides a comprehensive overview of all Python packages installed in the virtual environment used for the Team F Project, maintained within PyCharm. The environment was essential for backend development tasks, particularly involving Flask and FastAPI frameworks, as well as database and authentication tools.

The installed packages are listed in the table below, showing their names and corresponding versions. All packages were verified using the `pip list` command inside the virtual environment.

Installed Python Packages

Package Name	Version
annotated-types	0.7.0
anyio	4.9.0
bcrypt	4.2.0
blinker	1.8.2
click	8.1.7
colorama	0.4.6

fastapi	0.115.12
Flask	3.0.3
Flask-Bcrypt	1.0.1
flask-cors	5.0.1
Flask-Login	0.6.3
Flask-SQLAlchemy	3.1.1
Flask-WTF	1.2.2
greenlet	3.1.1
h11	0.14.0
idna	3.10
itsdangerous	2.2.0
Jinja2	3.1.4
MarkupSafe	3.0.2

pip	23.2.1
pydantic	2.11.1
pydantic_core	2.33.0
sniffio	1.3.1
SQLAlchemy	2.0.36
starlette	0.46.1
typing-inspection	0.4.0
typing_extensions	4.12.2
uvicorn	0.34.0
websockets	15.0.1
Werkzeug	3.0.6
WTForms	3.2.1